

EC 535 Introduction to Embedded Systems Final Project

“MEMs Mouse”

Devin Kot-Thompson

Link to Github Repo: <https://github.com/devinkt/ECE535FinalProject>

Link to Youtube Demo:

Abstract:

One of the most important and powerful considerations when designing technology is the human interaction with the device. The way we interact with our computers and phones dictates the positivity and productivity of the experience. For personal computers, mice and keyboards are the most popular interaction devices. For phones, capacitive touch screens have changed how we use cell phones. USB provides a powerful standard for Human Interfacing Devices (HIDs). This standard defines the way a USB host will recognize and interact with a HID device. It allows input and output functionality through the standard, without the need for custom device drivers [1]. This is an incredibly powerful tool for an embedded system with USB HID capability. It allows for the design of custom interfacing hardware to address different populations of need. Many individuals have limited mobility or fine motor control. They need custom designed hardware to interact with their computers. USB HID provides the capabilities needed to achieve this. This project was an exploratory investigation of using a USB HID capable processor to control a computer through an accelerometer sensor input.

Introduction:

The motivation for this project was to create a Custom USB HID device to control a computer using different inputs than a typical mouse or keyboard. USB HID is a powerful technology used in devices across industries. It is used in system controls, keyboards, mice, joysticks, webcams, medical instruments, sensors and much more [1]. This flexibility and ease of integration makes it a useful tool for computer assistive embedded devices. Common examples are eye tracking devices and sip and puff switches which allow disabled individuals to control their cursors and computers with limited mobility [2]. HID devices can help to improve the lives of these individuals and address hardware needs through the convenience of the USB standard. The hardware available for the development of this Custom USB HID device was a STM32F407G1-Discovery Evaluation Board. This board contains a STM32F407 microcontroller with an Arm Cortex M4 32b processor and integrated USB 2.0 OTG controller. USB OTG capable controllers are able to act as either USB host or USB devices[3]. This allows this board to be programmed into a HID device. Additionally, the Evaluation Board has a LIS3DSH accelerometer connected over a SPI bus and a tactile button [4]. The processor, peripheral, and sensor available on this evaluation board along with its IDE made it a good choice for my project of creating a custom HID device to investigate alternate methods of interfacing with computers.

Method:

To begin, a new project was created in STM32CubeIDE. STM32CubeIDE is STMicroelectronics IDE for their microcontroller products. It allows a convenient toolchain for your processor and a library management system. Additionally, there is a graphical peripheral management system (IOC) that allows code generation for system clock control and peripheral initialization. The generated code utilizes HAL (Hardware Abstraction Layer) which is ST's library for maintaining a low level bare metal control over microcontroller peripherals and clocks while abstracting the need to write to individual registers. The clock tree was set to use the external 8MHz crystal as the input clock source. The main PLL was set to generate a 72 MHz system clock and APB peripheral clocks, a 36 MHz peripheral clock, as well as a 48 MHz clock for the USB peripheral. The IOC also allows for the integration of middleware libraries including ST's USB Device library which allows for the generation of files supporting a USB HID device. Additionally, a USB OTG, SPI peripheral and several GPIO were initialized.

Once these initializations were complete the outline code was generated. This outline code includes the HAL drivers and the USB device middleware. The files that need to be configured for the custom HID device are the `usbd_customhid.c` and `usbd_custom_hid_if.c`. HID devices communicate with hosts through a packet of bytes called a report. ST provides a function to send reports to the host called `USBD_CUSTOM_HID_SendReport()`. It takes a handler to the USB device structure which has been initialized as a FS device, registered as a HID class, and registered with its interface operations. The registration of the HID class configures important device characteristics such as the device descriptor which describes the HID device to the PC host on enumeration. This is how the PC knows how often to poll the device for reports, how large of endpoints to reserve, and the size of the report descriptor. The registration of the interface operations provides access to the report descriptor as well as init, deinit and output event functions.

I created a custom report descriptor and modified the macros to initialize the correct report descriptor size and endpoint size allocation. The report descriptor is an array of bytes that are sent to the host that allow it to interpret the reports and the device type. The report descriptor has 2 Usage Pages, a keyboard and a mouse. The hierarchy of report descriptor bytes are organized into Usage Pages, Usages, and Collections. The Usage Page defines the general category of the device, the one in the project defines a Generic Desktop device. The Usage following the Usage Page defines what the device within that page will be recognized as by the host. This Usage is followed by a Collection which contains all the items of the device. These are the things that define the functionality of the device. These can include further sub usages, collections and usage pages. Additionally, it defines the reports of these functionalities and the values to be expected. For this device, the mouse has a button usage page. This gives a 1 bit absolute report either pressed or not with a 7 bit filler for byte aligned

reports. The mouse report also defines X and Y coordinates for the cursor in 16 bit relative measurements. The keyboard report has a report of 1 byte of modifier keys, 1 reserved byte, and 6 bytes of key presses. Additionally both Usage Pages send an identifier that the host knows so it can attribute the incoming report to the correct device.

Connecting this device to the PC, it enumerates in device manager as both a HID keyboard and mouse. The PC is polling the device every 5 ms for reports. To create input reports, a push button and LIS3DSH accelerometer were used. The LIS3DSH has configuration registers that can be written to using SPI

protocol. The HAL SPI functions were used to write a SPI driver for LIS3DSH. The LIS3DSH driver includes a Init function, Read function, Write function, Calibration function, Get Data function, State Machine Init function and State Machine Function. The LIS3DSH registers can be read from by sending 2 bytes over SPI. The first byte is the address of the register to read and the following byte can be another address or a garbage value. The second byte flushes data from the addressed register onto the data out line. To write to a LIS3DSH register, the address of a configuration register is sent followed by the configuration byte.

```

ALIGN_BEGIN static uint8_t CUSTOM_HID_ReportDesc_FS[USBD_CUSTOM_HID_REPORT_DESC_SIZE] __ALIGN_END =
{
    0x05, 0x01, // Usage Page (Generic Desktop)
    0x09, 0x02, // Usage (Mouse)
    0xA1, 0x01, // Collection (Application)
    0x85, 0x01, // Report ID 1
    0x09, 0x01, // Usage (Pointer)
    0xA1, 0x00, // Collection (Physical)
    0x05, 0x09, // Usage Page (Button)
    0x19, 0x01, // Usage Minimum (1)
    0x29, 0x01, // Usage Maximum (1)
    0x95, 0x01, // Report Count (1)
    0x75, 0x01, // Report Size (1)
    0x81, 0x02, // Input (Data,Var,Abs)
    0x95, 0x01, // Report Count (1)
    0x75, 0x07, // Report Size (7)
    0x81, 0x01, // Input (Const,Array,Abs)
    0x05, 0x01, // Usage Page (Generic Desktop)
    0x09, 0x30, // Usage (X)
    0x09, 0x31, // Usage (Y)
    0x16, 0x01, 0x80, // Logical Minimum (-32767)
    0x26, 0xFF, 0x7F, // Logical Maximum (32767)
    0x75, 0x10, // Report Size (16)
    0x95, 0x02, // Report Count (2)
    0x81, 0x06, // Input (Data,Var,Rel)

    0xC0, // End Collection

    // --- Keyboard report starts here ---
    0x05, 0x01, // Usage Page (Generic Desktop)
    0x09, 0x06, // Usage (Keyboard)
    0xA1, 0x01, // Collection (Application)
    0x85, 0x02, // Report ID 2
    0x05, 0x07, // Usage Page (Keyboard/Keypad)
    0x19, 0xE0, // Usage Minimum (Left Control)
    0x29, 0xE7, // Usage Maximum (Right GUI)
    0x15, 0x00, // Logical Minimum (0)
    0x25, 0x01, // Logical Maximum (1)
    0x75, 0x01, // Report Size (1)
    0x95, 0x08, // Report Count (8)
    0x81, 0x02, // Input (Data,Var,Abs) Modifier byte

    0x95, 0x01, // Report Count (1)
    0x75, 0x08, // Report Size (8)
    0x81, 0x01, // Input (Constant) Reserved byte

    0x95, 0x06, // Report Count (6)
    0x75, 0x08, // Report Size (8)
    0x15, 0x00, // Logical Minimum (0)
    0x25, 0x65, // Logical Maximum (101)
    0x05, 0x07, // Usage Page (Keyboard/Keypad)
    0x19, 0x00, // Usage Minimum (0)
    0x29, 0x65, // Usage Maximum (101)
    0x81, 0x00, // Input (Data,Array)

    0xC0 // End Collection
}

```

Fig 1. Report Descriptor

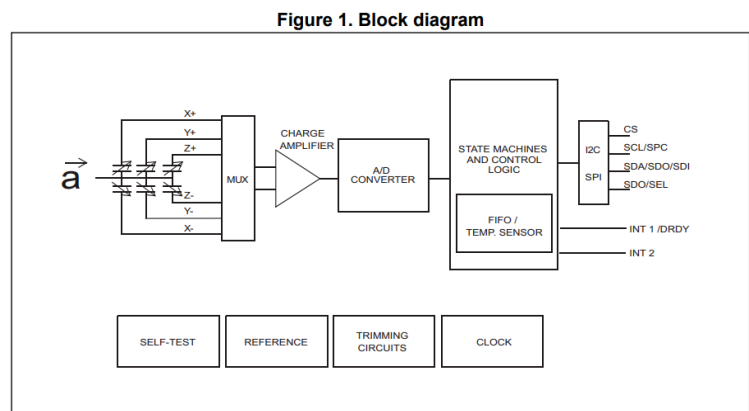


Fig 2. LIS3DSH Block Diagram

To configure LIS3DSH, the output data rate was set to 100 Hz. The scale was set to +/- 2g. The data output was set to continuous. The state machines and associated interrupts were enabled [5][6]. With this setup, X, Y, Z axis data could be read from the LIS3DSH and sent as reports to the USB host.

Additionally, an ITM trace was implemented to support debugging printouts.

Using these methods, a device can be created that can interface with a computer using various sensing modalities. These devices can be used for a wide range of applications due to the flexibility of the USB protocol and the support of STMCubeIDE's middleware. The system designed in this project provides an immersive interface for certain computer games.

Results:

The experimental testing of the device began by sending reports generated by the accelerometer and GPIO button to the USB host. This was done by holding the STM32 dev board and tilting it. The button functioned well as a left click of the cursor. However, the cursor itself was extremely sensitive to accelerometer movement and had drift. The ITM trace values gave important insight into what was happening. At rest, the X and Y axis still had small values (10-20 mg) likely from noise or a non-level surface. This small value was leading to drift and since the report values were 16 bit variance values there is already an inherent high

sensitivity to small values in the cursor space. To solve this problem, a calibration function was added before the device super loop. This calibration function reads axis values for a certain number of cycles calculating the minimum and maximum values continuously. These minimum and maximum values are then used in the data collection function to ensure small perturbations are canceled out. This helped to alleviate the drift after leaving the device flat during power up. To fix the sensitivity a simple scaling factor was added to the axis values. An additional step taken to ensure smooth cursor control

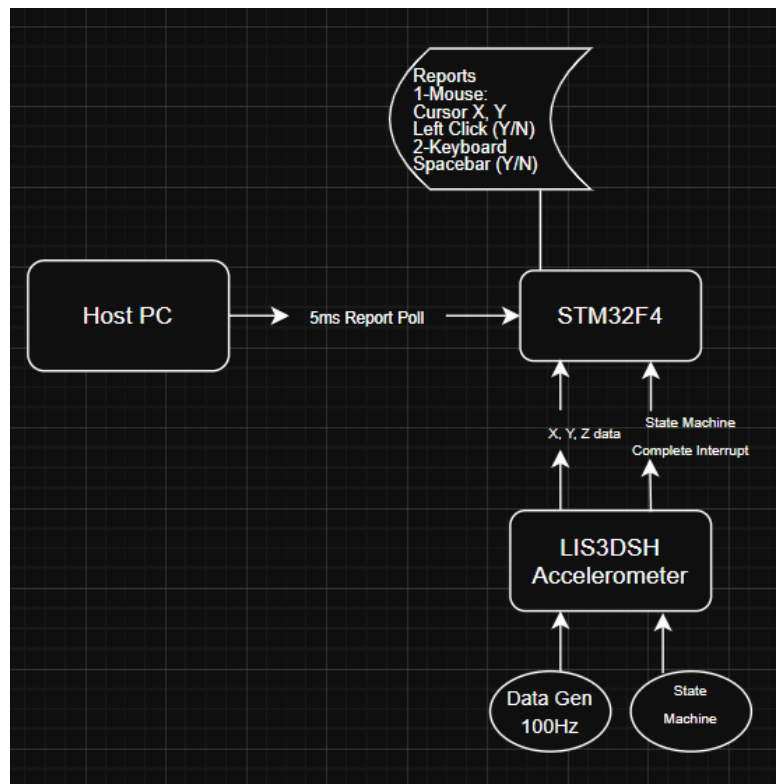


Fig 3. System Diagram

was to ensure the timing of reports aligned with the polling by the host. The accelerometer generates new data every 10 ms. The host is polling for reports every 5 ms. And 10 ms of delay are added using the HAL delay timer before and after each sent report in the super loop. This means the host is only seeing a variance change every 10 - 20 ms and is sending multiple 0 movement reports. After these steps, the cursor control was smooth as it now longer has drift, large sensitivity, and is sampling reports at an appropriate rate. The state machines inside the LIS3DSH allowed for keyboard inputs through gesture recognition in addition to the cursor control. These state machines work by configuring internal timers and threshold registers. A series of state machine stages are written to over SPI by address with a byte that sets the stage. The upper nibble is the reset condition and the lower nibble is the next condition. The conditions can be one of several opcodes. These machines can be configured to generate an interrupt when the final stage is reached or on exit [5][6]. For this device, a state machine was programmed over SPI to detect a single tap gesture. This used 5 stages. The first stage checks if any axis is below a configured threshold for a configured time. The second stage checks if a configured threshold is exceeded. The third stage checks if the axes drop below a configured threshold in a configured time. The fourth stage is a delay and then the 5th stage generates an interrupt. This interrupt was used to send a space bar pressed report to the host using the interrupt callback. To test the custom HID device, it was used to play a game called Agar.io. This game involves controlling a cell using the cursor and splitting the cell using the spacebar. The developed device was able to successfully play the game. Control was smooth and splitting the cell with simulated space bar input worked as intended.

Limitations and Future Work:

In the future, I would like to implement more gesture recognition using the accelerometer state machines. I would also like to investigate other sensor input modalities. Finally, it would be interesting to add output reports to a custom HID device for feedback from a program. Technical skills and knowledge I gained from this project include writing SPI drivers, working with USB HID devices, and using accelerometer data.

References:

[1]“HID Usage Tables FOR Universal Serial Bus (USB).” Available:

https://www.usb.org/sites/default/files/hut1_6.pdf

[2]“Types of assistive technology | Digital Accessibility,” *Berkeley.edu*, 2025.

<https://dap.berkeley.edu/web-a11y-basics/types-assistive-technology>

[3]Wikipedia Contributors, “USB On-The-Go,” *Wikipedia*, Feb. 20, 2025.

[4]“STM32F407/417 - PDF Documentation,” *STMicroelectronics*, 2025.

<https://www.st.com/en/microcontrollers-microprocessors/stm32f407-417/documentation.html>
(accessed May 01, 2025).

[5]“LIS3DH - STMicroelectronics,” *STMicroelectronics*, 2025.

<https://www.st.com/en/mems-and-sensors/lis3dh.html#documentation> (accessed May 01, 2025).